Final Assignment
Cryptography and Cyber Law
Md Joynal Abedin Joy
IT-21058

Question 1: How does shor's algorithm threaten the security of RSA and Elliptic curve cryptography (ECC), and what are the potential consequences for current digital infrastructure?

Answer:

Shor's algorithm: shor's algorithm, a quantum computing algorithm, poses a significant threat to the security of RSA and Elliptic curve cryptography (ECC) by efficiently solving the mathmatical problems these algorithms rely on. These systems ensure the confidentiality, authenticity, and integrity of everything from emails, to online banking and e-commerce.

shors algorithm poses a direct and grave threat to widely used in public-key crypto-graphy schemes like RSA and Elliptic curve cryphography (ECC) because it enables a quantum computer to efficiently solve the mathmatical problems- integer factorization (for RSA) and the elliptic curve discrete logarithm problem (for ECC)- that underlie their security in polynomical time instead of the exponential time required by classical algorithems.

RSA: Based on factoring large composite numbers, shor's algorithm uses quantum period finding and Quantum Fourier Trans- form to break RSA efficently.

ECC: Relies on the difficulty discrete logarithms on elliptic curves. shor's algorithm can solve this problem too, allowing

recovery of private keys from public ones. Estimates indicate Ecc may actually be easier to break than RSA in a quantum scenario: for 128-bit classical security, Ecc might require only ~ 2330 logical qubits versus ~ 4098 for RSA (though both are currently will beyond existing quantum hardware capabilities)

potential consequences for Digital Infrastructure

i. Breakdown of Internet Security:

- protocols like TLS/SSL (used in HTTP websites) will become insecure.
- Attackers can intercept and decrypt web traffic, exposing sensitive date (passwords, credit cards, etc).

(i. Data Vulnerability:

- Harvest Now, Decrypt later.
- Retrospective Decryption.
- Compromised confidentiality.

iii. Compromise of financial Systems

- Online banking and crypto currencies depend on RSA/ECC.
- Quantum computers could forge signatures, steal assets and manipulate financial transactions.

iv. Threat to Government and Military Data

- Encrypted data being collected today could be decrypted in the future by quantum computers.
- Long-term confidentiality of stored data is at severe risk.

v. Loss of Trust in Digital Identity and Signatures

- Quantum attacks can forge digital certificates and software signatures.

IT-21058

vi. Erosion of privacy:

- personal and sensitive data exchange though insecure networks could be expressed, leading to identity theft and privacy breaches.

vii. National Security Risks:

- Compromised government communications and vital services could house serious national security implications.

viii. Urgency for cryptographic Migration:

- If not addressed, there may be a sudden collapse in digital security often referred to as "Q-Day."

- Migrating to post-Quantum Cryptography (PQC) is essential to avoid widespread disruption.

shors algorithm is not just a the theoritical threat - it is ticking time bomb for all cymphophaphe systems based on RSA and ECC. Though large-scale quantam computen do not exist, their development is advancing nepidly once available they could instatly breale the foundations of current digital security.

Question 2: Discuss the role of quantum key distribution (QKD) in future emptographic systems. How does it diffen from classical public-key encryption?

Answer:

Quantum Key Distribution (QKD) is a secure commiunication method that utilises principles of quantum mechanics to generate and distributes emytrographic keys.

It allows two parties to create a shered secred key that can be used to encrypt and decrypt messeges, with security guanan-teed by the law of physics.

IT-21058

Role of QKD in Future Cryptographic Systems:

1. **Quantam-Safe key Exchange:**
   - Secure key sharing using quantam physics.
   - Resistant to quantum attacks.
2. **Unconditional Security:**
   - QKDs security is based on laws of quantam physics, such as:
     - Heisenberg's uncertainty principle
     - No-cloning theorem
   - Any attemp to eavesdrop on the quantum channel distrubs the system, alerting the legitimate parties.
3. **Eavesdropping Detection:**
   - Detects any interception via quantam distarbance.
   - Compromised keys are discarded.
4. **post-Quantum Ready:**
   - Used with symmetric encryption
   - Builds quantam-secure communication.

IT-21058

5. Quantum-Internet Backbones

- Core tech for future secure quantum networks.
- Already tested via satellites & fibe optics

6. physic-Based Trusts

- Security form laws of nature, not hard math.
- Not breakable by any computing power.

7. High security uses

- Best for military, banking, government.
- Long-term sensitive data protection.

8. Crypto-Agilety

- Frequent secure key updates.
- Adapts quickly to new threats.

IT-21058

Q How QKD Differs from classical public-key Encryption?

Feature	classical public key - Encryption (RSA, ECC)	Quantum key Distribution (QKD)
principle	Based on hard mathmatical problems.	Based on quantum physics
Security Type	Computational security	Information-theoretic security
Quantum Resistance	Vulnerable to quantum attacks	Resistant to quantum attacks by design.
Eavesdropping Detection	No built-in mechanism	Any interception disturbs the quantum state and is detectable
purpose	key exchange, encryption, digital signature	Secure key exchange only
Communication medium	classical channels (eg. internet, radio)	Quantum channels (eg. photons over fiber optics)

IT-21058

Maturity & Adoption	fully developed and widely used across digital system	Emerging technology limited to high-security applications.
Scalability and cost	scalable, low-cost implementation	High cost, limited range
Deployment	widely deployed worldwide	Experimental, critical systems only

Questions:3

What are the main differences between lattice basad cryptography and traditional number theoretic oppoaches like RSA, particularly in the context of quantum resistance?

Answer:

Lattice-based cryptography and traditional number-theoretic cryptography (such as RSA and ECC) differ significantly in their mathmatical foundations. In their mathmatical foundations security assumptions, and resistence

to quantum attacks.

Lattice based cryptography offers a significant advantage over traditional number theoretic approaches like RSA in terms of quantum resistance.

While traditional methods rely on problems that quantum computers can easily solve, lattice-based cryptography is built on problems that are believed to be difficult.

Here's more detailed breakdown of the differences:

1. Security foundation:

- Traditional (RSA/ECC)
  - Relies on the difficulty of factoring large numbers or solving the discreate logarithm problem.
- Lattice-based cryptographys
  - Relies on the difficulty of solving problems on high-dimensinal latti such as SVP and LWE.

IT-21058

2. Mathmatical foundations

— Traditional (RSA/ECC):
- Based on hard number-theoretic problems like Integer factorization (RSA) and discreate logarithm problems.

— Lattice-Based cryptography:
- Based on problems in high-dimensional geometry, such as the shortest vector problem (SVP) and LWE.

3. Quantum Resistance

— Traditional (RSA/ECC):
- Vulnerable to quantum algorithms like shor's algorithm, which can solve their underlying problems in polymonial time.

— Lattice-Based Cryptography:
- Considered quantum-resistant, as no efficient quantum algorithms are known to solve lattice problems.

4. Key size and performance:

— Traditional (RSA/ECC)

- use relatively smaller keys, but need very large keys for post-quantum security.

— Lattice-Based cryptography:

- use longer keys and ciphertexts, but many schemes are still efficient and scalable.

5. Maturity and standardization:

— Traditional (RSA/ECC)

- well studied and widely deployed for decades in protocols like TLS, VPN, SSH, etc.

— Lattice-Based cryptography:

- newer but gaining wide acceptance

6. Advanced Applications:

— Traditional (RSA/ECC):

- Mostly limited to encryption, signature and key exchange.

—Lattice-Based Cryptography:

- supports advanced techniques like homomorphic encryption, identity based encryption and post-quantum signatures.

Traditional number-theoretic cryptosystems like RSA are not secure in the quantum era as they can be broken by quantum algorithms. Lattice-based cryptography, on the other hand, is the foundation of future cryptographic systems. Hence, lattice-based schemes are considered a key solution for post quantum security.

Question 4: Develop a python based PRNG that uses the current system time and a custom seed value. Write complete programme and corresponding output.

Answer:

Here's a python PRNG (Pseudo-Random

Generator) that uses current system time and a custom seed value to generate random numbers. I'll also include sample output. So

phython program:-

```
import time
def custom-prng (seed, count):
    current-time = int (time.time-ns (1)
    combined-seed = seed ^ current-time
    a = 1664525
    B = 1013904223
    m = |2** 32|
    random-numbers = [ ]
    X = combined-seed
    for _ in range (count):
        X = (a*x+c) % m
        random-numbers. append (x)
    return random-numbers
seed-value = 12345
count = 5
numbers = custom-prng (seed value, count)
print ("custom PRNG output:")
for; num in enemenate (numbers, 1):
    print ( " Random Number {i}: {num}")
```

Sample Output:

Custom PRNG output:
Random Number 1: 1885951992
Random Number 2: 2910389135
Random Number 3: 2475739278
Random Number 4: 33994785
Random Number 5: 13263c4488

Explanation:

(i) seed initiazation → Combines custom seed and system time (nanoseconds) using XOR for randomness.

(ii) LCG Formula → uses $(a * X + c) \% m$ to generate psuedo-random numbers.

(iii) Quantum safety: This PRNG is not cryptographically secure but is suitable for simulation/random tasks.

(iv) Dynamic Output → Even with the same seed different times give different results.

IT-21058

Question:- Explain the sieve of Eratosthenes algorithm and use it to find all prime numbers less than 50. How does its time complexity compare to trial division?

Answer:

Sieve of Eratosthenes Algorithm:

The sieve of Eratosthenes is an ancient and efficient method for finding all prime numbers up to a given limit n. It works by progressively eliminating the multiples of each prime number, starting from the smallest prime(2).

Algorithm steps:

1. Create a list of integers from 2 to n
2. Start with the first numbers in the list (p=2), which is prime.
3. Eliminate all multiples of p.
4. Find the next number in the list that is not marked; this the next prime.

5. Repeat steps 3-4 until all multiples up to $\sqrt{n}$ have been processed.
6. The remaining unmarked numbers are all primes.

Final all primes less than 50

Let's use the sieve of Eratosthenes to find all prime numbers less than 50.

1. creat a list of numbers from 2 to 49:

2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17,
18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28,
29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 43, 44, 45, 46, 47, 48, 49

2. Start with the first prime, 2. then eliminate all multiplese of 2.

2, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27,
29, 31, 33, 35, 37, 39, 41, 43, 45, 47, 49

3. The next unmarked number 3. Eliminate all multiple of 3.

2, 3, 5, 7, 11, 13, 17, 19, 23, 25, 29, 31, 35,
37, 41, 43, 47, 49

IT-21058

4. The next unmarked number 5. Eliminate all multiple of 5:

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 49

5. The next unmarked number 7. Eliminate all multiple of 7:

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47

6. The next unmarked numbers is 11, and $11^2 = 121$, which is greater than 50. The algorithm stops here.

The remain unmarked numbers are the prime numbers less than 50:

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47.

Time complexity Comparison:

- Sieve of Eratosthenes: $O(n \log n)$ - very efficient for large ranges.

- Trial Division: $O(n\sqrt{n})$ - much slower for large n.

Thus, the sieve is significantly faster than trial division when generating all primes up to a large number.

Question 6: state and explain the necessary and sufficiant conditions for a composite number to be a carmichael number. Then verify whether the number $n = 561$ and $n = 1105$ and $n = 1729$ are carmichael numbers

Answer:

Definition: A carmichael number is a composite integer n such that

$$a^{n-1} \equiv 1 \pmod{n}$$

for every integer a with $\gcd(a, n) = 1$

A positive composite integer n is a carmichael number if all three conditions hold:

IT-21058

1. n is composite
2. n is square-free
3. for every prime p dividing n, (p-1) divides (n-1); i.e. $p-1 \mid n-1$

Verification for the given numbers:

We check each n against Korselt's criterion.

1. $n = 561$
   - Factorization: $561 = 3 \times 11 \times 17$ → composite and square free.
   - Compute $n-1 = 560$
   - For $p = 3$: $p-1 = 2 \cdot 2 \mid 560$ ($560/2 = 280$)
   - For $p = 11$: $p-1 = 10 \cdot 10 \mid 560$ ($560/10 = 56$)
   - For $p = 17$: $p-1 = 16 \cdot 16 \mid 560$ ($560/16 = 35$)

All conditions satisfied → 561 is a carmichael number.

2. $n = 1105$
   - Factorization: $1105 = 5 \times 13 \times 17$ → composite and square-free
   - Compute $n-1 = 1104$

- For $p=5$: $p-1 = 4 \cdot 4 \mid 1104$ $(1104/4 = 276)$
- For $p=13$: $p-1 = 12 \cdot 12 \mid 1104$ $(1104/12 = 92)$
- For $p=17$: $p-1 = 16 \cdot 16 \mid 1104$ $(1104/16 = 69)$

All conditions satisfied → 1105 is a carmichael number.

Ex: $n = 1729$

- Factorization: $1729 = 7 \times 13 \times 19$ → composite and square-free.
- Compute $n-1 = 1728$
  - For $p=7$: $p-1 = 6 \cdot 6 \mid 1728$ $(1728/6 = 288)$
  - For $p=13$: $p-1 = 12 \cdot 12 \mid 1728$ $(1728/12 = 144)$
  - For $p=19$: $p-1 = 18 \cdot 18 \mid 1728$ $(1728/18 = 96)$

All conditions are satisfied → 1729 is a carmichael number.

7. Determine whether the following are valid algebraic structures and justify your answer:
   - Is the set Z_{11} with operations $(+, \cdot)$ a ring?
   - Are the sets $(Z_{37}, +)$ and $(Z_{35}, \times)$ Abelian groups?

IT-21058

Answer:

Yes, In fact Z_{11} is a commutative ring with unity and moreover a field.

justification:

- $(Z_{11}, +)$ is an abelian group: closure, associatively, identity 0, inverses ($-a \equiv 11 - a$) and commutativity hold (addition mod 11)
- Multiplication mod 11 is closed, and associative, and distributes over addition.
- There is a multiplicative identity $1 \in Z_{11}$.
- Because 11 is prime, every nonzero element $a \in Z_{11}$ has a multiplicative inverse mod 11, Thus all ring axioms hold and every nonzero element is invertible → Z_{11} is a field.

Yes, $(Z_{37}, +)$ is a Abelian group.

justification:

- Set $Z_{37} = \{0, 1, \ldots, 36\}$ with addition modulo 37.

- closure, associativity and commutativity follow from integer addition.
- Identity is 0. For each a, additive inverse is 37-a.
- Thus $(Z_{37}, +)$ satisfies all group axioms and is abelian.

No, $(Z_{35}, \times)$ is not a group.

Justification:

- Although multiplication mod 35 is associative and has identity 1, not every element has a multiplicative inverse in Z_{35}.
- Example: $5 \in Z_{35}$. $\gcd(5, 35) = 5 > 1$.
  There is no x with $5x \equiv 1 \pmod{35}$ because any product $5x$ is divisible by 5 and cannot be congruent to 1. Hence 5 has no inverse.
- Therefore the inverse axiom fails and $(Z_{35}, \times)$ is not a group, so it is not an abelian group.

IT-21058

Question 8: what is the remainder when -52 is reduced modulo 31?

Answer:

we want r with $0 \leq r < 31$ and $-52 \equiv r \pmod{31}$

Compute: $-52 + 2 \cdot 31 = -52 + 62 = 10$

so, $-52 \equiv 10 \pmod{31}$

alternate: $-52 \equiv -52 + 31 = -21 = -21 + 31 \equiv 10$

so, the remainder is 10

Question: 9: Determine the multiplicative inverse of 7 mod 26, if it exists. (use extended Euclidean algorithm)

Answer:

we solve $7x \equiv 1 \pmod{26}$ or find integers x, y with $7x + 26y = 1$

use the Euclidean algorithm:

$$26 = 3 \cdot 7 + 5,$$
$$7 = 1 \cdot 5 + 2,$$
$$5 = 2 \cdot 2 + 1,$$
$$2 = 2 \cdot 1 + 0.$$

Back-substitute to express 1 as a linear combination:

$$1 = 5 - 2 \cdot 2$$
$$= 5 - 2(7 - 1 \cdot 5) = 3 \cdot 5 - 2 \cdot 7$$
$$= 3(26 - 3 \cdot 7) - 2 \cdot 7 = 3 \cdot 26 - 11 \cdot 7$$

Thus $-11 \cdot 7 + 3 \cdot 26 = 1$. Therefore $x \equiv -11 \pmod{26}$

$$x \equiv -11 \equiv 15 \pmod{26}.$$

The multiplicative inverse of 7 modulo 26 is 15.

Question: 10

Evaluate $(-8 \times 5) \bmod 17$, and explain how to simplify negative modular multiplication.

Answer:

step 1 - Multiply the numbers:

$$-8 \times 5 = -40$$

step 2 - Reduce modulo 17:

we find the equivalent positive remainder:

$$-40 + 3 \times 17 = -40 + 51 = 11$$
$$-40 \equiv 11 \pmod{17}$$
$$(-8 \times 5) \bmod 17 = 11$$

IT-21058

Explain of simplification method:

Negative numbers in modular arithmetic can be converted to their positive equivalent before multiplying.

$$-8 \equiv 9 \pmod{17}$$

$$9 \times 5 = 45$$

Reduce 45 modulo 17:

$$45 - 2 \times 17 = 45 - 34 = 11$$

This match the privious result

∴ final answer -11

Question 11: state and prove Bezouds Theorem use it to find the multiplicative inverse of 97 modulo 385.

Answer:

For integers a and b not both zero, there exist integers x, y such that

$$ax + by = \gcd(a, b)$$

In particular, if $\gcd(a, b) = 1$ there are integers x, y with $ax + by = 1$; then x is the multiplicative inverse of a modulo b.

proof:

Let, $d = \gcd(a,b)$

- By the Euclidean algorithm, d divides both a and b.
- The set of all integers of the form $an+by$ is closed under addition and substraction.
- Let me be the smallest positive number is this set. Then m divides both a and b.
- Hence $m=d$, and there exist integers x, y such that:

$$ax+by=d$$

Finding the multiplicative inverse of 97 modulo 385:

we want x such that:

$$97x \equiv 1 \pmod{385}$$

This is equivalant to:

$$97x + 385y = 1$$

we use the Extended Euclidean Algorithm:

M-21058

1. Euclidean algorithm:

$$385 = 97 \cdot 3 + 94$$
$$97 = 94 \cdot 1 + 3$$
$$94 = 3 \cdot 31 + 1$$
$$3 = 1 \cdot 3 + 0$$

So, $\gcd(97, 385) = 1$

2. Back substitution:

From $94 = 3 \cdot 31 + 1$

$$1 = 94 - 3 \cdot 31$$

from $97 = 94 \cdot 1 + 3$

$$3 = 97 - 94$$

Substitute into 1:

$$1 = 94 - (97 - 94) \cdot 31$$
$$1 = 94 - 97 \cdot 31 + 94 \cdot 31$$
$$1 = 94 - (1+31) - 97 \cdot 31$$
$$1 = 94 \cdot 32 - 97 \cdot 31$$

From $385 = 97 \cdot 3 + 94$

$$94 = 385 - 97 \cdot 3$$

Substitute:

$$1 = (385 - 97 \cdot 3) 32 - 97 \cdot 31$$
$$1 = 385 \cdot 32 - 97 \cdot 96 - 97 \cdot 31$$
$$1 = 385 \cdot 32 - 97 (96 + 31)$$
$$1 = 385 \cdot 32 - 97 \cdot 127$$

3. Conclusion:

we have:

$$1 = (-127) \cdot 97 + 32 \cdot 385$$

Therefore, $x \equiv -727 \pmod{385}$

science we want the positive inverse:-

$$-127 \equiv 385 - 127 = 258 \pmod{385}$$

The multiplicative inverse of 97 mod 385 is 258.

Question 12: Sing Bezoud's identity prove that the equation $ax+by = \gcd(a,b)$ has integer solutions. Find x such that $43x \equiv 1 \bmod 240$.

Answer:

Bezout Identity statement:

For any integers a and b, there exist integers x and y such that:

$$ax+by = \gcd(a,b)$$

proof:

1. Let $d = \gcd(a,b)$ By definition, d divides, d divides both a and b, so $a = da'$ and $b = db'$, where a', b' are integers with $\gcd(a',b') = 1$

2. science a' and b' are comprime, there exists integers x_0, y_0 such that :-

$$a'x_0 + b'y_0 = 1.$$

3. Multiplying though by d gives :

$$a(dx_0) + b(dy_0) = d$$

Thus $x = x_0$ and $y = y_0$ are integer solutions to : $ax + by = \gcd(a, b)$

Find x such that $43x \equiv 1 \pmod{240}$:

This asks for the multiplicative inverse of 43 modulo 240. we solve the Diophantine equation.

$$43x - 240y = 1$$

by the Extended Euclidean Algorithm.

Compute gcd chain (Euclidean algorithm) :

$$240 = 5 \cdot 43 + 25,$$

$$43 = 1 \cdot 25 + 18,$$

$$25 = 1 \cdot 18 + 7,$$

$$18 = 2 \cdot 7 + 4,$$

$$7 = 1 \cdot 4 + 3,$$

$$4 = 1 \cdot 3 + 1,$$

$$3 = 3 \cdot 1 + 0.$$

So. gcd (43, 240) = 1 and the inverse exist.
Now Back-substitute to express 1 as a linner combination of 43 and 240.

Back-substitution:

$$1 = 4 - 1 \cdot 3$$
$$= 4 - 1(7 - 1 \cdot 4) = 2 \cdot 4 - 17$$
$$= 2 \cdot (18 - 2 \cdot 7) = 2 \cdot 18 - 5 \cdot 7$$
$$= 2 \cdot (8 - 5(25 - 1 \cdot 18) = 7 \cdot 18 - 5 \cdot 25$$
$$= 7 \cdot (43 - 1 \cdot 25) - 5 \cdot 25 = 7 \cdot 43 - 12 \cdot 25$$
$$= 7 \cdot 43 - 12(240 - 5 \cdot 43)$$
$$= 7 \cdot 43 - 12 \cdot 240 + 60 \cdot 43$$
$$= 67 \cdot 43 - 12 \cdot 240$$

Thus, $43 \cdot 67 - 240 \cdot 12 = 1$,

So, $x = 67$ is a solution

$$43^{-1} \equiv 67 \pmod{240} \text{ or } 43 \cdot 67 \equiv 1 \pmod{240}$$

Question 13: prove Fermat's little Theorem and explain how it is used to test for primality. Is 561 a prime number based on this test?

Evaluate $5^{123} \bmod 175$ using fermet's little Theorem. show all steps.

SIT21058

Answer:

Fermat's Little Theorem (FLT) - statement:

If p is a prime number and a is any integer with gcd (a,p) = 1, then

$a^{p-1} \equiv 1 \pmod{p}$.

proof:

Consider the nonzero residues modulo p: 1, 2 ..... p-1 multiply each by a (with gcd (a,p)=1)

The set $a \cdot 1, a \cdot 2, \ldots a \cdot (p-1)$

is a permutation of $1, 2 \ldots p-1$ modulo p.

Taking the product of all elements in both sets and reducing mod p gives

$$a^{p-1} = (1 \cdot 2 \cdots (p-1) \equiv 1 \cdot 2 \cdots (p-1) \pmod{p}$$

cancelling the nonzero product $1 \cdot 2 \cdots (p-1)$ which is invertible mod p) yields $a^{p-1} = 1 \pmod{p}$

primality Test using FLT:

choose a with $1 < a < p$ and gcd $(a,p) = 1$

If $a^{p-1} \not\equiv 1 \pmod{p} \Rightarrow p$ is composite. If it is 1, p may be prime - but there are composites (carmicheel numbers) that also pass.

IT-21058

Is 561 prime?

Take $a = 2$; $\gcd(2, 561) = 1$;

$561 = 3 \cdot 11 \cdot 17$

By chinese Remainder Theorem and FLT:

$2^{560} \equiv 1 \pmod 3, 1 \pmod{11}, 1 \pmod{17}$

So, $2^{560} \equiv 1 \pmod{561}$ - 561 passes FLT But is not prime. It's a carmichael number)

Evaluate $5^{123} \bmod 175$

1. Factor the modulus: $175 = 25 \cdot 7$ work modulo 25 and 7 combine by CRT.
2. Mod 25: $5^2 = 25 \Rightarrow 5^n \equiv 0 \pmod{25}$ for all $n \geq 2$ since $123 \geq 2$

$5^{123} \equiv 0 \pmod{25}$

3. Mod 7: $\gcd(5, 7) = 1$ so by Fermat $5^6 \equiv 1 \pmod 7$ Reduce the exponent.

$123 \equiv 123 - 6 \cdot 20 = 3 \pmod 6$;

$5^{123} \equiv 5^3 = 125 \equiv 6 \pmod 7$

4. Combine by CRT: find x with

$x \equiv 0 \pmod{25}$, $x \equiv 6 \pmod 7$.

check multiples of 25: $25 \cdot 5 = 125 \equiv 0 \pmod{}$

Thus the solution modulo 175 is

$$5^{128} \equiv 125 \pmod{175}.$$

Fermats little Theorem cannot be applied directly modulo 175, because 5 and 175 are not coprime - thats why we used CRT.

Question 4c state and prove the chinese Remainder Theorem. Then solve the following system of congruences.

$x \equiv 2 \pmod{3}, x \equiv 3 \pmod{5}$

$x \equiv 2 \pmod{7}$

Answer:

Statement: The chinese Remainder Theorem states that a system of linear congruences with pairwise coprime module has a unique solution modulo the product of the module.

If $m_1, m_2, \ldots, m_k$ are pairwise compmime positive integers and $a_1, \ldots, a_k$ are any integers,

Then the system

$x \equiv a_i \pmod{m_i}$, $(i = 1, \ldots, k)$

has a unique solution modulo $M = m_1 m_2 \cdots m_k$

proof (constructive):-

Let $M = \prod_{i=1}^{k} m_i$ and $M_i = M/m_i$ since $\gcd(M_i, m_i) = 1$, there exists an inverse y_i such that $M_i y_i \equiv 1 \pmod{m_i}$ Then

$$x = \sum_{i=1}^{k} a_i M_i y_i$$

satisfies $x \equiv a_j \pmod{m_j}$ for each j (because for $i \neq j$ M_i is divisible by m_j). Uniqueness modulo M follows because any two solutions differ by a multiple of all m_i, hence by a multiple of M.

Solve the system:

stard with $x \equiv 2 \pmod 3$ put $x = 2 + 3k$

plug into mod 5:

$$2 + 3k \equiv 3 \pmod 5 \Rightarrow 3k \equiv 1 \pmod 5$$

Inverse of 3 mod 5 is 2 (since $3 \cdot 2 = 6 \equiv 1$), so,

$k \equiv 2 \pmod 5 \Rightarrow k = 2 + 5t$

Thus $x = 2 + 3k = 2 + 3(2 + 5t) = 8 + 15t$

Now impose mod 7:

$8 + 15t \equiv 2 \pmod 7 \Rightarrow 15t \equiv 6 \equiv 1 \pmod 7$

since $15 \equiv 1 \pmod 7$, this gives $t \equiv 1 \pmod 7$

So, $t = 1 + 7s$, Then

$x = 8 + 15(1 + 7s) = 8 + 15 + 105s = 23 + 105s$

Therefore the solution modulo $3 \cdot 5 \cdot 7 = 105$

$x \equiv 23 \pmod{105}$

Question 15: Briefly explain the CIA triad in information security. How does each component contribute to building a secure system?

Answer:

The CIA triad stands for confidentiality, Integrity and Availability. It's a fundamental for information security.

IT-21058

- **Confidentiality:**
  - Ensures data is disclosed only to authorized parties.
  - Mechanisms: encryption, access controls, authentication, secure channels
  - Contribution: prevents data leaks, protects privacy and secrets.
- **Integrity:**
  - Ensures data is accurate and has not been tampered with.
  - Mechanisms: cryptographic hashes, digital signatures, MACs, checksums, version control.
  - Contribution: detects and prevents unauthorized modification, supports trust in data and transactions.
- **Availability:**
  - Ensures authorized users can access data and services when needed.

- Mechanisms: redundancy, backups; DDOS protection, fault tolerance, monitoring.
- Contribution: keeps systems usable and resilient; prevents service distruption which could harm business operations.

All three are required together: strong confidentiality with no availability is useless availability without integrity/confidentiality is risky. Good security design balances CIA based on system requirements.

Question 14: How does steganography differ from cryptography in the context of information security, and what are common techiques used for hiding data in Digital media?

Answer:

Difference: steganography vs cryptography

- cryptography transforms plaintext into

an unreadable form (ciphertext) so that the content is hidden (confidential), but the existence of the message is usually the existen ;abious. Good: Unreadability without the key.

- Steganography: hides the existence of the message by embedding it into another innocuous subject (image, audio, video, text)

Goal: secrecy of existence (covert communication).

They can be used together. first encrypt the secret, then hide the ciphertext with steganography.

Common steganography techniques (digital media)

1. LSB (Least significant Bit) substitution (image/audio): replace least significant bits of pixels/samples with message bits simple and high capacity but vulnerable to processing/noise.

2. Masking and diftering (images): hide data in perceptually significant areas (watermarking style) robust against image processing.

3. Transform-domain methods: embaded data in frequency coefficients. More robust to compression and common edits.

4. spread-spectrum steganography: spread message bits across many samples, low detectability and robust.

5. protocol/metadata hiding: store message in file metadata, unused header fields, or network protocols fields.

6. Text steganography: using whitespace, synonyms, or formating changes.

Question IX: what are the key difference between phishing malware and denial-of-service (DoS) attacks in terms of their method and impack on system security?

IT-21058

Answers

phishing:

- Method: social engineering - attackers send deceptive emails/messages or create fake website to trick users into revealing credentials, personal data, or clicking malicious links.
- Goal/Impact: Credential theft, identity theft, initial foothold in networks.
- Defences: User education, email filtering multi, factor authentication (MFA). URL/websites scanners, anti phishing policies.

2. Malware

- Method: software (viruses, worms, trojans, ransomware, spyware) delivered via email attachments, drive-by-downloads, infected devices, or malicious installers.
- Goal/Impact: Data theft, destruction, system compromise, encryption for ransom, establishing persistent backdoors.

- Defences: Endpoint protection / AV; application white listing, patch management, least privilege, network segmentation, breakups.

3. Denial-of-service (DOS/DDOS):

- Method: Overwhelm a service or network with traffic or exploit resource limits.
- Goal/Impact: Availability loss - legitimate users cannot access service, can cause business disruption and revenue loss.

Defence: Rate limiting, traffic filtering, DDOS mitigation services (scrubbing) content delivery networks, redundancy.

Comparison summary:

- phishing targets people
- Malware targets systems/software to steal, modify or destroy data

IT-21058

Question 3.8: Explain how legal frameworks such as the General Data protection Regulation (GDPR) help mitigate cyber attacks and protect user privacy.

Answer:

Legal frameworks and cybersecurity: Role of GDPR:

The General Data Protection Regulation (GDPR) is a European union (EU) law designed to strengthen the protection of personal data and privacy of individuals. It plays an important role in mitigating cyber attacks and protecting user privacy though the following ways.

1. Data Minimization:- GDPR requires organizations to collect only necessary personal data, reducing the amount of sensitive information that can be stolen in a cyber-attack.

IT-21058

2. Security Measures:- It mandates strong technical and organizational measures to safeguard data against unauthorized access, alteration, or loss.

3. Breach Notification:- Organizations must report personal data breaches to authorities within 72 hours. This ensures quick containment and response to cyber attacks.

4. User Rights:- Individuals have rights such as access to their data correction, deletion and data probability.

GDPR significantly reduces the risk and impact of cyber attacks, while ensuring the privacy and trust of users.

IT-21058

Question 19: Explain the basic working of the DES algorithm using a simple 64-bit plaintext block and a 56-bit key. Show how the initial permutation, round functions, and final permutation contribute to the encryption process.

Answer:

Basic working of the DES Algorithm:

The Data Encryption standard (DES) is a symmetric-key block cipher that encrypts data in fixed-size blocks.

It operates on:

- plaintext block : 64 bits (8 bytes)
- key : 56 bits
- output : 64-bit ciphertext

1. Initial permutation (IP)

- The 64-bit plaintext is rearranged according to a fixed table.

- This step does not change the key; it simply changes bit positions to prepare for the rounds.
- Example: If the plaintext is $P = [P_1, P_2 \cdots P_{64}]$, the IP might move P_{58} to position 1, P_{50} to position 2, etc.

2. Rounds (16 Iterations)

The output of IP is divided into two halves:

- Left half (L0): first 32 bits.
- Right half (R0): Last 32 bits.

a) Expansion (E-box)

- The 32-bit right half (R) is expanded to 48 bits by duplicating certain bits, using the E expansion table.

b) Key Mixing

- A 48 bit subkey is generated from the main 56-bit key, using a key schdule.
- The expanded R is XORed with the subkey: R-expanded ⊕ Subkey

IT-21058

c. Substitution (S-boxes)

- The 48-bit result is divided into 8 blocks of 6 bits.
- Each block is passed though a S-box to produce a 4-bit output.
- Total output = 32 bits.

d. Permutation (P-box):

- The 32-bit S-box output is permuted using a fixed P-box table.

e. Swap and combine:

- The output from P-box is XORed with the left half (L)
- The right half becomes the new left half and the result of the XOR become the new right half.

Formulas for each rounds:

$$Ln = Rn-1$$

$$Rn = Ln-1 \oplus f(Rn-1, Kn)$$

3. Final Permutation (FP)

- After 16 rounds, the final swap is performed.
- The two halves are joined and passed through the inverse of the Initial permutation to produce the ciphertext.

Question 2.05 : In the DES algorithm, a 64-bit plaintext block is divided into two 32-bit halves: L_0 and R_0. Given $R_0 = 0xF0FDFUFD$ and round key $K_1 = 0x0F0F0F0F$, compute the output of the first rounds function $f(R_0, K_1)$ assuming XOR operation only. Then, find $L_1 = R_0$ and $R_1 = L_0 \oplus f(R_0, K_1)$, where $L_0 = 0xAAAAAAAA$.

Answere

Given: $R_0 = 0xF0F0F0F0$

Round key $K_1 = X_0 0F0F0F0F$

$L_0 = 0xAAAAAAAA$

IT-21058

Step 1:- Compute $f(R_0, k_1)$ assuming XOR operation only.

$f(R_0 k_1) = R_0 \oplus k_1 = 0xF0F0F0F0 \oplus 0xF0F0F0F0$

perform XOR byte-wise:

- $F0 \oplus 0F = FF$
- $F0 \oplus 0F = FF$
- $F0 \oplus 0F = FF$
- $F0 \oplus 0F = FF$

So, $f(R_0, k_1) = 0xFFFFFFFF$

Step 2: Compute $L_1 = R_0 = X_0F0F0F0F0$

Step 3: Compute $R_1 = L_0 \oplus f(R_0, k_1)$

$R_1 = 0xAAAAAAAA \oplus 0xFFFFFFFF$

XOR, byte-wise:

- $AA \oplus FF = 55$
- $AA \oplus FF = 55$
- $AA \oplus FF = 55$
- $AA \oplus FF = 55$

So, $R_1 = 0x55555555$

$\therefore L_1 = 0xF0F0F0F0$, $R_1 = 0x55555555$

(Answer)

IT-21058

Question 21. Given the input word

[0x23, 0xA7, 0x4C, 0x19]

use the following partial AES s-box to perform the subbytes transformation provide the resulting output word.

Row↓ \ col→	3	4	7	9	A	C
1	6D	•	•	C6	•	•
2	D4	•	•	•	•	•
4	•	A1	•	•	•	2E
A	•	•	63	•	D2	•

Given Input word:

[X023, 0xA7, 0x4C, 0x19]

partial AES s-box table:

Row↓ | col→	3	4	7	9	A	C
1	6D	•	•	C6	•	•
2	D4	•	•	•	•	•
4	•	A1	•	•	2E	•
A	•	•	63	D2	•	•

IT-21058

For each byte, the high nibble (4 bits) is the row and the low nibble is the coloumn.
Lets find the corresponding s-box value for each byte:

- For 0x23 : row = 2, col = 3 → from table, row 2 col 3 = D4
- For 0xA7 : row = A, col = 7 → From table, row A col 7 = 63
- For 0x9C : row = 4, col = C → From table, row 4 col C = (not given in table → dot), so we leave it as is or unknown.
- For 0x19 : row = 1, col = 9 → from table, row 1 col 9 = C6

Resulting output word:

[0xD4, 0x63, unknown, 0xC6]

Since 0x9C is not present, we indicate unknown or no mapping given.

1T-21058

Question 22: In AES encryption, apply the AddRoundkey step only Given:

- Input word: [0x1A, 0x2B, 0x3C, 0x4D]
- Round key word: [0x55, 0x66, 0x77, 0x88]

perform XOR between the input word and the round key, and write the resulting output word.

Answere

The AddRoundkey step in AES encryption invole performing a bitwise XOR operation between the input word and the round keyword.

Given:

input word: [0x14, 0x2B, 0x3C, 0x4D]

Round keyword: [0x55, 0x66, 0x77, 0x88]

Compute the output word (XOR each byte)

Solution (byte-wise XOR):

0x1A ⊕ 0x56 = 0x4f (00011010 + D1 01011 = 01001111)

0x2B ⊕ 0x66 = 0x4D (00101011 ⊕ 01100110 = 01001101)

IT-21058

$0x3C \oplus 0x77 = 0x4B$ $(00111100 \oplus 01110111 = 01001011)$

$0x4D \oplus 0x88 = 0xC5$ $(01001101 \oplus 10001000 = 11000101)$

Resulting output words:

$[0x4F, 0x4D, 0x4B, 0xC5]$

Brief explanation: The AddRoundKey step performs a bitwise XOR between each byte of the round key, producing the output word shown above.

Question 23: show how MixColumns uses the following fixed matrix over GF(2^8):

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

to transform an input column. Use an example column with values $[0x01, 0x02, 0x03, 0x04]$.

[convert hex to Binary]

IT-21058

Answers:

The Mixcolumns operation is AES works by multiplying each column of the state matrix by a fixed matrix over the Galois field GF(2^8)

Given that,

Input column (bytes):

$$a = \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 0x01 \\ 0x02 \\ 0x03 \\ 0x04 \end{bmatrix}$$

Mixcolumns Matrix:

$$M = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

So, the output column is $b = M.a$ where each entry is bytewise GF(2^8) arithmetic (XOR for addition)

Multiplication rules used (AES)

- Multiply by 01 : value itself.
- Multiply by 02 (called x time) : left shift by 1 bit; if orginal MSB = 1 then XOR with 0x1B after the shift.
- Multiply by 03 : $03 \cdot x = 02 \cdot x \oplus 01 \cdot x$

IT-21058

Input bytes in binary:

- $0x01 = 00000001_2$
- $0x02 = 00000010_2$
- $0x03 = 00000011_2$
- $0x04 = 00000100_2$

Row 1 calculation

$$b_0 = (02 \cdot a_0) \oplus (03 \cdot a_1) \oplus (01 \cdot a_2) \oplus (01 \cdot a_3)$$

Compute terms:

→ $02 \cdot a_0 = 02 \cdot 0x01 = x\text{time}(0x01) = 0x02$

→ $03 \cdot a_1 = [(02 \cdot 0x02) \oplus 0x02] = x\text{time}(0x02) \oplus 0x02$

$0x02 = 0x04 \text{ XOR } 0x02 = 0x06$

→ $01 \cdot a_2 = 0x03$

→ $01 \cdot a_3 = 0x04$

Now XOR them:

$$b_0 = 0x02 \oplus 0x06 \oplus 0x04 = (0x02 \oplus 0x06) = 0x04)$$

$$\oplus 0x04 \oplus 0x03 \oplus 0x04 = 0x04 \oplus 0x03$$

$$\oplus 0x04 = 0x03$$

So, $b_0 = 0x03$

Row 2 calculations

$$b_1 = (01 \cdot a_0) \oplus (02 \cdot a_1) \oplus (03 \cdot a_2) + (01 \cdot a_3)$$

IT-21058

Terms:

- $01 \cdot a_0 = 0x01$
- $02 \cdot a_1 = x\text{time}(0x02) = 0x04$
- $03 \cdot a_2 = x\text{time}(0x03) \text{ XOR } 0x03 = (x\text{time}(0x03) = 0x06) \text{ XOR } 0x03 = 0x05$
- $01 \cdot 03 = 0x04$

XOR:

$b_1 = 0x01 \oplus 0x04 \oplus 0x05 \oplus 0x04 = (0x01 \oplus 0x04 = 0x05) \oplus 0x05 \oplus 0x04 = 0x00 \oplus 0x04 = 0x04$

So, $b_1 = 0x04$

Row 3 calculations:

$b_2 = (01 \cdot a_0) \oplus (01 \cdot a_1) \oplus (02 \cdot a_2) \oplus (03 \cdot a_3)$

Terms:

→ $01 \cdot a_0 = 0x01$

→ $01 \cdot a_1 = 0x02$

→ $02 \cdot a_2 = x\text{time}(0x03) = 0x06$

→ $03 \cdot a_3 = x\text{time}(0x04) \text{ XOR } 0x04 = 0x08 \text{ XOR } 0x04 = 0x0c$

XOR:

$b_2 = 0x01 \oplus 0x02 \oplus 0x06 \oplus 0x0c$

IT-21058

Compute stepwise:

$0x01 \oplus 0x02 = 0x03$; $0x03 \oplus 0x06 = 0x05$;

$0x05 \oplus 0x0C = 0x09$

So, $b_2 = 0x09$

Row 4 calculation:

$$b_3 = (03 \cdot a_0) \oplus (01 \cdot a_1) \oplus (01 \cdot a_2) \oplus (02 \cdot a_3)$$

Terms:

- $03 \cdot a_0 = x\text{time}(0x01) \text{ xor } 0x01 = 0x02 \text{ xor } 0x01 = 0x03$
- $01 \cdot a_1 = 0x02$
- $01 \cdot a_2 = 0x03$
- $02 \cdot a_3 = x\text{time}(0x04) = 0x08$

xor:

$$b_3 = 0x03 \oplus 0x02 \oplus 0x03 \oplus 0x08$$

stepwise:

$0x03 \oplus 0x02 = 0x01$; $0x01 \oplus 0x03 = 0x02$;

$0x02 \oplus 0x08 = 0x0A$

So, $b_3 = 0x0A$

Final result (output column)

$$b = \begin{bmatrix} 0x03 \\ 0x04 \\ 0x09 \\ 0x0A \end{bmatrix}$$

Binary forms—

- $0x03 = 00000011_2$
- $0x04 = 00000100_2$
- $0x09 = 00001001_2$
- $0x0A = 00001010_2$

Question 246 Describe how AES OFB mod works, How does it ensure synchronization between encryption and decryption streams?

Answere—

AES-OFB (Output Feedback) is a mode of operation for the Advanced Encryption Standard (AES) that turns a block cipher into a stream cipher. It works by generating a keystream, which is then XORed with the plaintext to produce the ciphertext.

IT-21058

How it works:

1. Initialization: The process starts with a unique initialization vector (IV). which is typically a random value. This IV is fed into the AES block cipher.

2. Keystream Generation: The output of the AES encryption of the IV becomes the first part of the keystream. This output is also fed back into the AES block cipher for the next round, generating the next part of the keystream. This continues, creating a sequence of keystream blocks.

3. Encryption: Each block of the keystream is xored with a corresponding block of plaintext to produce the ciphertext.

4. Decryption: The decryption process is identical to the encryption process. The same IV is used to generate the exact same key-

stream. The keystream is then xored with the ciphertext to recover the original plaintext. The key to OFB mode is that the keystream is generated independently of the plaintext and ciphertext.

Synchonization in AES-OFB:

- Both sender and recieven stand with the same initialization vector (IV) and same secret key.
- keystream blocks are generated only from the previous keystream block, not from the ciphertext or plaintext.
- This means if IV and key are identical, both sides produce exactly the same keystream sequance, keeping encryption and decryption aligned.

If IV or key changes, synchonization will break.

IT-21058

Question 25: Which AES modes cause error propagation during decryption and how does this affect the integrity of the decrypted message? Illustrate with CBC and CFB modes.

Answere

AES Modes causing Error propagation:

Some AES modes ~~Error propagation~~ cause an error in a ciphertext block to affect multiple plaintext blocks during decryption. This reduces the integrity of the decrypted message because even a single-bit error can corrupt more that one block.

1. CBC (Cipher block chaining) mode:

- Decryption formula: $P_i = AES^{-1}(c_i) \oplus c_{i-1}$
- ~~Effect~~ effect:

→ if c_i has a 1-bit error → P_i becomes completely random.

→ the same bit position in P_{i+1} will also be flipped.

IT-21058

- Error affects two consecutive plaintext blocks.

2. CFB (cipher feedback) Mode

- Decryption formula : $P_i = C_i \oplus AES(C_{i-1})$
- Error effect :
  - If C_i has a 1-bit error → the same bit in P_i is wrong.
  - P_{i+1} becomes completely random.
  - After that, decryption returns to normal.

Question 26 : which AES mode would you recommend for encrypting large files with parallal processing, and why? Justify your choice between ECB, CBC and CT.

Answer:

Recommended modes: AES-CTR (counter-mode)

IT-21058

Justifications

- parallel processing:
  - In CTR, each block is encrypted by XORing the plaintext with a keystream generated from AES (key, counter)
  - The counter values for all blocks are known in advance, so encryption and decryption of different blocks can happen independently and in parallel.
- performance:
  - No chaining between blocks - faster than CBC for large files.
- Security:
  - Unlike ECB, CTR does not produce identical ciphertext for identical plaintext blocks.
  - As secure as CBC when IV/nonce is unique.

IT-21058

- Flexibility:
  - Works efficiently for both stream-like and random access data.

Why not ECB?

- Reveals patterns in the data - insecure for large files.

Why not CBC?

- Each block depends on the previous ciphertext block → cannot be parallelized in encryption.

For large file encryption with parallel processing AES-CTR is best because it is secure and highly parallelizable.

Question 27: Given a message "A" represented as M = 14, encrypt it using public key e=5, n=74. What is ciphertext? Then, decrypt using private key d=11.

IT-21058

Solution:

Given, $m=1$, $e=5$, $n=14$, $d=11$

Encryption: $c \equiv m^e \bmod n = 1^5 \bmod 14 = 1$.

Decryption: $m \equiv c^d \bmod n = 1^{11} \bmod 14 = 1$

Why this works: RSA relies on $m^{ed} \equiv m \pmod{n}$ when $ed \equiv 1 \pmod{\varphi(n)}$. Here, $\varphi(14)=6$ and $e \cdot d = 5 \cdot 11 = 55 \equiv 1 \pmod 6$, so decryption recovers m.

∴ Ciphertext $c=1$ and Decrypted message $m=1$

Question 28: Given message hash: $H(m)=35$, RSA private key: $d=3$, $n=33$, Generate the digital signature.

Answer:

Given: hash $H(m)=5$, $d=3$, $n=33$

Signature generation (Sign with private key d):

We need to generate a digital signature for a message hash:

IA-21058

- The message hash is $H(m) = 5$
- The RSA private key is $(d, n) = (3, 33)$
- The digital signature is is computed using the formula:

$$S = H(m)^d \pmod{n}$$

$$S \Rightarrow 125 \pmod{}$$

- $125 = 3 \times 33 + 26$ so $125 \pmod{33} = 26$
- The Digital signature is 26.

Question: 29c Aleya and Badol are using the Diffie Hellman key exchange protocal. They agree on the following public values: prime modulus : $p = 17$, Base (generation): $g = 3$, Aleya chooses a private key, $a = 4$, Badol chooses a private key $b = 5$. compute public key of Aleya and Badal.

Answer:

We'll compute the public keys for aleya and Badol based on the Diffie-Hellmen protocol.

1T21058

- public values: prime modulus $p = 17$, base $g = 3$.
- Aleya's private key: $a = 4$
- Badal's private key: $b = 5$
- Aleya's public key (A):
  - $A = g^a \pmod p = 3^4 \pmod{17} = 81 \pmod{17}$
  - $81 = 4 \times 17 + 13$ so $81 \pmod{17} = 13$
- Aleya's public key is 13
- Badal's public key (B):
  - $B = g^b \pmod p = 3^5 \pmod{17} = 243 \pmod{17}$
  - $243 = 14 \times 17 + 5$, so $243 \pmod{17} = 5$
- Badal's public key is 5

Question 30: A simple (non-cryptographic) hash function $H(x)$ is defined as the sum of ASCII values of the characters in a message, modulo 100:

$$H(x) = (\Sigma \text{ASCII of characters in } x) \bmod 100$$

Compute the hash value of the message "A" and "BA" using this function. Do the two messages produce the same hash? What does this imply about collision resistance in weak hash functions?

IT-21058

Answer:

Definition:

$H(x) = (\sum ASCII(chars\ in\ x)) \bmod 100$

ASCII values: A = 65, B = 66

compute hashes:

$H("AB") = (65+66) \bmod 100 = 131 \bmod 100 = 31$

$H("BA") = (66+65) \bmod 100 = 131 \bmod 100 = 31$

Result:

Both "AB" and "BA" produce the same hash → collision

collision Resistance:

- The two different messages "AB" and "BA" produce the same hash value. This is called collision.
- This hash function is not collision-resistant because it is based on a simple modulansum.

IT-21058

Question 31: In a secure message system a simple message Authentication code (MAC) is computed using modular addition;

MAC = (message + secretkey) mod 17

Given: message is 15 secret key: 7. compute the MAC for the message. suppose an attacker changes the message to 10 bit doesn't know the key. can they forge the correct MAC easily? Explain Briefly.

Answer:

Given, message M = 15, K = 7

Compute MAC:

MAC = (15+7) mod 17 = 22 mod 17 = 5

Attacker scenario: suppose attacker changes message to M' = 10 but does not know K.

Read MAC M' is:

MAC' = (10+7) mod 17 = 17 mod 17 = 0

IT: 21058

Why forging is easy (Explain):

Because MAC is linear: MAC = M+K (mod 17)
if attacker knows one valid pair (m, MAC), and wants to create m' = m+Δ, they can compute MAC' = MAC + Δ (mod 17) without knowing K.

Example here: Δ = -5 ≡ 12 (mod 17);

MAC' = 5+12 = 17 ≡ 0, which matches the true MAC.

Conclusion: This MAC is insecure it is trivially forgeable because it leaks linear relation with the message.

Question 32: Explain the steps involved in the TLS handshake process. How are symmetric keys established securely using asymmetric cryptography during the handshake?

Answer:

TLS Handshake steps and symmetric key Establishment:

IT-21058

1. client-Hello → client sends supported TLS version, ciphen suites, random numbers.
2. ServenHello → servers selects TLS version, and cipher suite, sends its random number.
3. Servers certificate → Contains servers public key, signed by CA for Authentication,
4. Key exchange →
   - RSA mode
   - ECDHE / DHE mode.
5. Pre-Master secret → Master secret.
6. Session key → From master secret.
7. Finished messages → Both parties send encrypted "finished" message to confirm the handshake.

How symmetric key are established securely-
Asymmetric encryption (RSA or Diffie-Hellman) esures that only the intended parties can compute the shered secret from which symmetric keys are derived.

Question 33: Explain the layered architecture (protocol stack) of SSH. Briefly describe the roles of each layer.

Answer:

SSH refers to "secure shell" is a protocol that provides a secure channel over an unsecured network. It has a layered architecture consisting of three main layers.

1. Transport layer protocol:

This is the lowest layer, responsible for managing the secure connection.

- Handles encryption
- Integrity protection
- Compression
- Server authentication.

2. User Authentication protocol:

This layer runs on top of the transport layer and handles client authentication

- Verifies the client's identity using

password. public keys or other methods.

3. Connection protocol:

Multiplexes the encrypted channel into multiple logical channels. This is the highest layer.

Question 34: Explain the steps involved in the TLS handshake process.

Answer: TLS handshake process step:

1. clientHello → purposes connection parameters
2. serverHello → proposes connection parameters.
3. certificate and key exchange.
4. Generate Master secret
5. Generate session keys
6. Finished messages.
7. Secure communication

IT-21058

Question 35: What is the general form of all elliptic curve equation over a finite field, and why is it used in cryptography?

Answer:

The general form of an elliptic curve equation over a finite field is:

$$y^2 = x^3 + ax + b \pmod{p}$$

Here, p is a large prime number that defines the finite field; and a and b are constants such that

$$4a^3 + 27b^2 \neq 0 \pmod{p}$$

(ensures no singularities)

Use in cryptography:

It provides a group structure for Elliptic curve cryptography (ECC), enabling secure key exchange, signature and encryption with smaller keys.

1T-21058

Question 36: How does Ecc achieve the same level of security as RSA with a smaller key and size? Briefly explain.

Answer:

Elliptic curve cryptography (ECC) is a public key cryptography technique that provides the same cryptographic strength as RSA but with much smaller key sizes. The main reason lies in the underlying mathematical problems.

1. Mathematical Foundation:

- RSA is based on the Integer factorization problem (IFP)
- Ecc is based on the Elliptic curve Discrete lagarithm problem (ECDLP)

2. Key size comparison:

- Ecc 256-bit ≈ RSA 3072-bit security level
- Ecc 384-bit ≈ RSA 7680-bit security-level

3. Conclusions

ECC provides strong security with reduced key sizes due to the computational hardness of the ECDLP compared to integer factorization.

Question 37: Given the elliptic curve $y^2 = x^3 + 2x + 3 \pmod{97}$, determine whether the point $P = (3, 6)$ lies on the curve.

Answers

Given that,

Curve: $y^2 \equiv x^3 + 2x + 3 \pmod{97}$

point: $P = (3, 6)$

$LHS = y^2 = 6^2 = 36 \pmod{97}$

$RHS = x^3 + 2x + 3 = 27 + 6 + 3 = 36 \pmod{97}$

Since L.H.S = R.H.S point lies on the curve

IT-21058

Question 38: Given public key ($p=23, g=5, h=8$) and message $m=10$, compute the ElGamal ciphertext using random $k=6$.

Answer:

To compute the ElGamal ciphertext, we use the following public values:
$p=23$, $g=5$, $h=8$, and the message $m=10$, The random number is $k=6$

The ciphertext consists of two parts (c_1, c_2)

- c_1 computation:

$$c_1 = g^k \pmod p$$

$$c_1 = 5^6 \pmod{23}$$

$$5^2 = 25 \equiv 2 \pmod{23}$$

$$5^4 = (5^2)^2 \equiv 2^2 = 4 \pmod{23}$$

$$5^6 \equiv 5^4 \times 5^2 \equiv 4 \times 2 = 8 \pmod{23}$$

So, $c_1 = 8$

IT-21058

c_2 computations

$c_2 = m \times h^k \pmod{p}$

First, we calculate h^k:

$h^k = 8^6 \pmod{23}$

$8^2 = 64 = 2 \times 23 + 18 \equiv 18 \pmod{23}$

$8^4 = (8^2)^2 \equiv 18^2 = 328 = 14 \times 23 \times 2 \equiv 2 \pmod{23}$

$8^6 = 8^4 \times 8^2 \equiv 2 \times 18 = 36 = 12 \times 23 + 13 \equiv 13 \pmod{23}$

Now, we compute c_2:

$c_2 = 10 \times 13 \pmod{23} = 130 \pmod{23}$

$130 = 5 \times 23 + 15 \equiv 15 \pmod{23}$

So, $c_2 = 15$

The ElGamal ciphertext is $(8, 15)$

$$\boxed{c_1, c_2 = (8, 15)}$$